

PathFinder — Security Documentation

1. Security Overview

PathFinder implements a defense-in-depth security strategy with multiple layers of protection across the application stack. This document details every security mechanism in the system.

2. Authentication

2.1 Password Security

- **Hashing Algorithm:** `bcrypt`
- **Salt Rounds:** 10
- **Storage:** Only the hashed password is stored in the database. Plaintext passwords are never persisted.
- **Comparison:** `bcrypt.compare()` is used for login verification, which is timing-safe.

2.2 JWT (JSON Web Tokens)

- **Signing Algorithm:** HS256 (HMAC with SHA-256)
- **Payload:** Contains `userId` and `role` only (minimal claims).
- **Expiry:** 24 hours from issuance.
- **Secret Key:** Stored in environment variable `JWT_SECRET`. Never hardcoded in production.
- **Storage:** Stored in the browser's `localStorage` via `AuthContext`.

2.3 Password Reset Flow

1. User requests a password reset with their email.
2. System generates a cryptographically secure random token: `crypto.randomBytes(32).toString('hex')`.
3. Token and its expiry (1 hour from now) are saved to the `User` record.
4. Reset link is sent via email (Nodemailer with Ethereum test accounts in development).
5. Upon reset, the token is verified against the database and checked for expiry.
6. After successful reset, the token is nullified in the database.

Anti-Enumeration Protection: The forgot-password endpoint always returns the same success message regardless of whether the email exists, preventing attackers from determining valid email addresses.

3. Authorization

3.1 Role-Based Access Control (RBAC)

Three roles with strict permission boundaries:

Role	Can Access
STUDENT	Student profile, applications, AI tools, bookmarks, notifications, tickets (own)
ORGANIZATION	Org profile, opportunities (own), applicants (own), notifications, tickets (own)
ADMIN	All of the above + admin analytics, user management, all tickets, audit logs, system health

3.2 Middleware Enforcement

Two middleware functions protect routes:

1. **`authenticate`:** Extracts the JWT from the Authorization header, verifies it, and attaches the decoded user to the request object. Returns 401 if the token is missing or invalid.
2. **`requireRole(role)`:** Checks that the authenticated user's role matches the required role. Returns 403 if insufficient permissions.

3.3 Resource-Level Authorization

Beyond role checks, many endpoints verify resource ownership:

- Students can only view/modify their own profile and applications.
- Organizations can only manage their own opportunities and applicants.
- Ticket viewing is restricted to the creator and admins.

- Application timelines verify that the requester is either the student or the organization owner.
 - Admins cannot delete their own account (prevents logout).
-

4. API Security

4.1 Rate Limiting

- Applied to `/api/auth` routes.
- Configuration: 5 requests per 15-minute window per IP address.
- Response when exceeded: HTTP 429 with message "Too many requests, please try again later."
- Purpose: Prevents brute-force login attacks and credential stuffing.

4.2 CORS (Cross-Origin Resource Sharing)

- Configured to allow only the frontend origin.
- Methods restricted to: GET, POST, PUT, DELETE.
- Credentials are enabled for cookie support.

4.3 Input Validation

- All request bodies are parsed as JSON via `express.json()` middleware.
 - Prisma ORM uses parameterized queries, preventing SQL injection.
 - File uploads are validated by extension and size before processing.
-

5. File Upload Security

5.1 Upload Restrictions

Constraint	Value
Maximum file size	5MB
Allowed extensions	.pdf, .doc, .docx, .png, .jpg, .jpeg
Storage location	/uploads directory (not publicly served)

5.2 Filename Sanitization

Uploaded files are renamed using a unique pattern: `{fieldname}-{timestamp}-{random}.{extension}`. Original filenames are never used on disk, preventing path traversal attacks.

5.3 Access Control

- **Images** (.png, .jpg, .jpeg): Publicly accessible via `/api/files/:filename` (used for profile photos and logos).
 - **Documents** (.pdf, .doc, .docx): Require JWT authentication via the `authenticate` middleware before serving.
-

6. Data Protection

6.1 Sensitive Data Handling

- Passwords: bcrypt-hashed, never logged or returned in API responses.
- JWT Secret: Environment variable only.
- AI API Keys: Environment variable only, never sent to the frontend.
- Reset Tokens: Cryptographically generated, time-limited, single-use.

6.2 Database Security

- SQLite database file (`dev.db`) is excluded from version control via `.gitignore`.
 - All foreign key relationships use `onDelete: Cascade` to prevent orphaned records.
 - User deletion cascades to all related data (GDPR-like data removal).
-

7. Security Best Practices

1. **Environment Variables:** All secrets are stored in `.env` files which are excluded from Git.
2. **Minimal JWT Claims:** Tokens contain only `userId` and `role` — no sensitive data.
3. **Error Messages:** Generic error messages are returned to clients to prevent information leakage.
4. **No Stack Traces:** Server errors are caught and logged server-side only; clients receive generic 500 responses.

5. **Token Expiry:** 24-hour JWT expiry limits the window of opportunity for stolen tokens.
6. **HTTPS:** Both Vercel and Render enforce HTTPS in production.

End of Security Documentation